Project overview

Portfolio Lens | Stock Analyzer App

Project documentation • Version 1.0 • October 11, 2026

Executive summary

Portfolio Lens is a local Streamlit application that turns a stock transaction CSV into a consolidated portfolio view, historical transaction analysis and evidence-based AI explanations. It combines deterministic FIFO accounting with Yahoo Finance daily closing prices and an optional OpenAI assistant.

Problem and objective

A transaction spreadsheet records purchases and sales but does not directly explain remaining holdings, cost basis, current allocations or the impact of partial sales. The project brings these views together and allows the user to ask questions using the same calculations shown in the dashboard.

Project attribute	Version-one scope
Primary user	An individual reviewing their stock portfolio
Input	Complete transaction history uploaded as CSV
Account assumption	One U.S. taxable brokerage account
Accounting	USD; FIFO; stocks only
Interface	Four Streamlit tabs
AI role	Explain supplied transactions and calculated results
Storage	Session memory; no portfolio database

Core design decision

The application calculates financial results in Python. OpenAI explains those results. This separation makes the accounting testable and keeps the dashboard useful when AI credentials, API credits or network access are unavailable.

This document describes the implemented local prototype. It does not claim production readiness, certified tax accuracy or guaranteed error-free model responses.

User experience and input data

Tab	User experience
1. Import portfolio	Upload and preview transactions, review validation warnings, and confirm the date through which account history is complete.
2. Portfolio overview	View remaining shares, original and adjusted basis, closing prices and dates, market value, allocations and economic gains.
3. Historical performance	Select a period; review monthly purchases/proceeds, realized results, daily holdings value, FIFO disposals and wash-sale findings.
4. Ask your portfolio	Ask natural-language questions, follow up on earlier answers, and inspect the evidence sent to OpenAI.

CSV contract

Column	Expected value
ticker	Stock symbol, such as AAPL
date	Trade date in YYYY-MM-DD format
transaction_type	BUY or SELL
quantity	Positive, finite share quantity
price	Positive, finite per-share price in USD

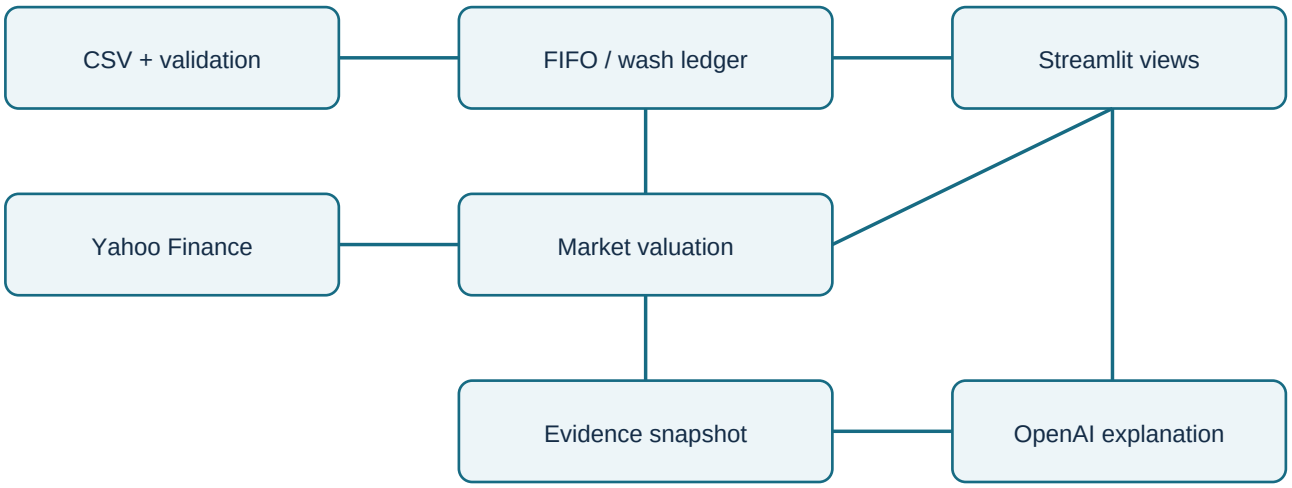
Example: AAPL,2024-01-03,BUY,10,185.07. The required header is ticker,date,transaction_type,quantity,price.

Import rules

Each valid upload replaces the active dataset. Dates are sorted chronologically; CSV row order determines same-day ordering. CSV row numbers become transaction IDs. Invalid rows, future trades and sales exceeding available shares reject the import while preserving the prior valid dataset.

Identical rows produce a warning and are retained because identical trades can be legitimate. The file must include the history needed to establish all purchase lots; opening balances and inherited prior wash adjustments are not supported.

High-level system design



Data flow

1. The upload layer parses and validates the CSV into typed Trade objects. 2. The ledger reconstructs FIFO lots and processes sales and replacement-share matches. 3. The market layer fetches Yahoo histories when requested and joins valid closing prices to holdings. 4. Streamlit renders the results. 5. On an opted-in question, the AI layer sends a current evidence snapshot and bounded conversation history to OpenAI.

Component boundaries

Financial computation has no dependency on a language-model response. Price failures leave affected values unavailable rather than substituting transaction prices. AI failures show a diagnostic and do not create an answer.

State and service behavior

Transactions, results and chat are managed in the user’s Streamlit session. There is no persistent portfolio database. Yahoo price histories use a 15-minute cache; the refresh button clears the cache before refetching. Portfolio records are not placed in that global cache.

The app runs as a local Python process. Ending the process or losing the session can require a fresh import. External dependencies are Yahoo Finance for market data and the OpenAI API for optional explanations.

Low-level code structure

File or module	Responsibility and key functions
app.py	UI orchestration, four tabs, session state, date filters, caching controls and chat. cached_prices() wraps the market fetch.
ledger.py	Trade and Lot models. parse_csv() normalizes input; analyze() reconstructs FIFO, economic basis, adjusted basis, sales and wash matches.
market.py	fetch_history() requests and checks Yahoo data; value_holdings() computes current valuations; historical_values() builds the daily holdings-value series.
assistant.py	answer() packages evidence and calls the Responses API; error_message() provides safe diagnostics; IRS_URL and RULES anchor general wash-sale explanations.
tests/	test_ledger.py, test_integrations.py and test_app.py cover accounting examples, service boundaries and Streamlit rendering.
pyproject.toml / uv.lock	Python dependencies and reproducible environment.
.env / .gitignore	Local API configuration; secret files are excluded from Git.

Internal models and outputs

Trade contains the CSV row ID, ticker, date, side, Decimal quantity and Decimal price. Lot carries its purchase Trade, remaining quantity, adjusted per-share tax cost and replacement-matching flag. Partial sales and partial matches split the relevant quantities.

analyze() returns holdings, sales, wash_sales, lot_disposals, total purchases and sale proceeds. Tables and charts use float representations; lot arithmetic uses Decimal without intermediate cent rounding.

Technology choices

Streamlit supplies the interactive UI; pandas supports tables and time series; yfinance retrieves Yahoo data; the OpenAI Python SDK calls the Responses API; python-dotenv loads local configuration. LangChain packages exist in the assignment environment but are not used by the current app.

Financial logic and verified sample

Metric	Definition
Remaining quantity	Shares bought minus shares sold
Market value	Remaining shares × latest available daily close
Allocation	Ticker market value ÷ complete holdings value × 100
Realized economic gain	Sale proceeds minus original FIFO cost of shares sold
Unrealized economic gain	Market value minus original cost of remaining shares
Combined economic gain	Realized economic gain plus unrealized economic gain
Tax-adjusted realized gain	Proceeds minus adjusted FIFO cost plus disallowed wash loss

Sample results: transaction-derived, no live price assumptions

Ticker	Shares	Original basis	Adjusted basis
AAPL	15	$2,907.85	$2,983.05
MSFT	9	$3,715.43	$3,715.43
TSLA	4	$721.24	$721.24

The supplied CSV contains 18 transactions from January 3, 2024 through June 5, 2026. Total purchases are $18,630.79; sale proceeds are $11,455.29. Realized economic gain is $169.02. Estimated tax-adjusted realized gain is $244.22 under the implemented single-account screening assumptions. These totals span the full dataset, not one tax year.

Worked wash-sale example

On June 4, 2026, five AAPL shares sell for $876.15. FIFO assigns the May 21 purchase lot with original cost $951.35, producing a $75.20 loss. The engine matches five remaining June 1 replacement shares, the earliest eligible purchase, and increases their per-share basis from $190.27 to $205.31. This sale’s estimated tax-adjusted gain is $0.00.

Coverage is marked provisional unless the user confirms history through July 4, 2026 or later. A covered window refers only to this uploaded account. IRS Publication 550 describes the inclusive 30-day-before/after window and replacement-share matching order [1].

AI design, grounding and data handling

How an AI answer is generated

The app sends an explicit evidence snapshot containing transactions, holdings, sale-lot details, wash-sale matches, valuations, coverage information and price errors. The snapshot is accompanied by the question, grounding instructions and up to eight recent conversation messages. The default model is gpt-4.1-mini and can be changed in the UI or environment.

The implementation uses the OpenAI Responses API with store=False. It does not use an autonomous tool-selection loop, vector database or retrieval-augmented document index. The current IRS reference is supplied as curated text with a source link; there is no live tax-law browsing during a chat answer [2].

Questions supported by existing evidence

Examples: “How much have I realized from MSFT?”, “Which holding has the largest unrealized gain?”, “Explain the AAPL wash-sale finding”, and “Why do my economic and tax-adjusted results differ?” Questions that require absent data should be identified as unavailable.

Grounding controls

Instructions require supplied calculated figures, supporting transaction IDs and dates, and a distinction between economic and tax estimates. Input and chat content are treated as untrusted data. A visible evidence inspector lets the user review the payload. These measures reduce unsupported answers but cannot guarantee that the model will never make a mistake.

Data handling and diagnostics

API keys stay in the local .env file. Yahoo receives symbols and date ranges. OpenAI receives portfolio evidence only when the user enables sharing and asks a question. store=False disables response storage for this request; it does not override other provider data policies.

The UI distinguishes exhausted API credits/quota, invalid keys, model access errors, connection failures, timeouts and rate limits without echoing raw request bodies or secrets. Hypothetical trade simulation is not implemented; the assistant must not claim to have run one.

Verification and operating guide

Verification performed

The most recent completed test run passed all 14 tests. Coverage includes sample reconciliation, FIFO partial sales, partial future replacements, an outside-window purchase, prior replacement splitting, chained losses, incomplete coverage, invalid inputs, missing quotes, price carry-forward without backfill, safe billing/key diagnostics, a mocked AI request and a four-tab Streamlit smoke test.

A live Yahoo check returned seven AAPL daily price rows. OpenAI model-access verification succeeded. A subsequent minimal response-generation check returned credit_balance_exhausted; a successful live AI explanation has not yet been verified. Automated tests avoid billable AI generation.

Start the application

```
cd Week1-assignment
uv sync
uv run streamlit run app.py
```

Open the URL printed in the terminal, normally http://localhost:8501. Keep the terminal process running. Click Load sample portfolio or upload a CSV, then fetch prices in the sidebar.

Configure optional AI

```
OPENAI_API_KEY=your-key
OPENAI_MODEL=gpt-4.1-mini
```

Add these entries to the local .env file. Use a valid API project with available credits and model access. Restart Streamlit after key changes, enable portfolio sharing in the AI tab and ask a question. No real key is included in this document.

Run verification

```
uv run python -m unittest discover -s tests -v
```

Common operational issues

If the browser cannot connect, check that Streamlit is running. If valuation is unavailable, review the ticker error and price dates. If AI reports exhausted credits, resolve API billing before retrying. A server restart can clear session data; reload the portfolio.

Presentation walkthrough and roadmap

Suggested five-minute demonstration

1 minute — Introduce the problem and scope: one taxable account, USD, FIFO and stocks only. Load the sample and show validation and completeness controls.

1 minute — Fetch prices and review shares, basis and allocation. Point out the price dates and explain that daily closes are not a real-time quote feed.

1 minute — Select a historical period and show monthly spending, sale proceeds and FIFO details. Explain that the holdings-value chart includes the impact of buys and sells and is not a percentage-return chart.

1 minute — Show the June 4 AAPL wash-sale finding and the $75.20 adjustment. Explain the separate economic and tax views and account coverage limits.

1 minute — If API billing is available, ask an evidence-based question and show its supporting transaction details. Otherwise demonstrate the evidence inspector and describe the request flow without presenting a fabricated answer.

Current boundaries

No cash ledger, dividends, commissions, deposits/withdrawals, account-level percentage return, multi-account or spouse matching, IRAs, options, shorts, cross-ticker substantially identical securities, tax filing export or long/short-term classification. Holding-period carryover is not implemented. Stock splits detected by Yahoo block valuation pending reconciliation; other corporate actions may require manual review.

Proposed roadmap — not implemented

Phase 2: cash and cash flows, dividends/fees and a defined return methodology. Phase 3: persistent portfolios, account metadata and corporate-action reconciliation. Phase 4: broader tax-lot reporting, holding periods and reviewable exports. Phase 5: deterministic hypothetical trade simulation and more selective AI evidence retrieval.

References

[1] IRS Publication 550 (2025), Wash Sales: https://www.irs.gov/publications/p550

[2] OpenAI Responses API: https://developers.openai.com/api/reference/python/resources/responses/methods/create

[3] yfinance history documentation: https://ranaroussi.github.io/yfinance/reference/api/yfinance.Ticker.history.html

Implementation evidence: app.py, src/week1_assignment/, tests/, README.md and Week1 Sample Data - Sheet1.csv. Sample tax figures assume confirmed history coverage and are screening estimates, not filing records.